

SERVO VOLTAGE STABILIZER

DC BASED USING DIODES AS MULTIRANGE RELAY SWITCHING RANGES

ABSTRACT

This project presents the design and implementation of a smart servo-based voltage stabilizer integrated with real-time monitoring, adaptive control, and instrumentation features. The system utilizes a microcontroller to regulate and maintain stable output voltage by selecting between multiple voltage sources (12V and 24V) using relay-based switching. A servo mechanism is incorporated to represent dynamic control action, enhancing the visualization of stabilization.

The stabilizer continuously monitors input voltage, output voltage, and system noise using appropriate sensing circuits. A precision reference voltage generated using a 7805 regulator is employed for calibration, improving measurement accuracy and compensating for system variations. The system also includes noise detection to analyze power quality, making it more advanced than conventional stabilizers.

A MOSFET-based power control stage enables smooth and controlled power delivery to the load through PWM, implementing soft-start and efficient load management. The design supports both automatic and manual modes for stabilization and load control, allowing flexibility and adaptability for different operating conditions.

User interaction is simplified using a single potentiometer and a multi-purpose push button, implementing a context-based control interface to manage multiple functions efficiently. Visual and audible indicators such as LEDs and a buzzer provide real-time system status, including stability, load condition, and fault alerts.

Unlike traditional stabilizers, this system not only maintains voltage stability but also incorporates monitoring, calibration, and intelligent control, making it suitable for modern power management applications. The project demonstrates an integrated approach combining power electronics, embedded systems, and instrumentation techniques for improved reliability, efficiency, and performance..

Table of Contents

CHAPTER 1:-

- 1.1 INTRODUCTION.....**
- 1.2 HISTORY.....**
- 1.3 PROJECT AIM.....**
- 1.4 CHALLENGES.....**

CHAPTER 2:-

- 2.1 PROCEDURE.....**
- 2.2 DESIGN SPECIFICATIONS.....**

CHAPTER 3:-

- 3.1 LIST OF COMPONENTS.....**
- 3.2 DESCRIPTION OF EACH COMPONENT.....**
- 3.3 WORKING OF THE CIRCUIT.....**

CHAPTER 4:-

- 4.1 SOURCE CODE.....**
- 4.2 SOFTWARE FLOW.....**
- 4.3 CODE EXPLANATION.....**
- 4.4 ADVANTAGES OF SOFTWARE DESIGN.....**

CHAPTER 5:-

- 5.1 BLOCK DIAGRAM.....**
- 5.2 BLOCK DIAGRAM DESCRIPTION.....**
- 5.3 CIRCUIT DIAGRAM.....**
- 5.4 CIRCUIT DIAGRAM DESCRIPTION.....**

CHAPTER 6:-

- 6.1 ADVANTAGES.....**
- 6.2 APPLICATIONS.....**
- 6.3 LIMITATIONS.....**
- 6.4 FUTURE SCOPE.....**

CHAPTER 7:-

- 7.1 FINAL CONCLUSION.....**

CHAPTER 1

1.1 INTRODUCTION:-

A voltage stabilizer maintains a constant output voltage to protect devices from fluctuations that can cause damage or inefficiency. Conventional stabilizers use basic relay and transformer systems, but they lack accuracy, adaptability, and monitoring. This project introduces a smart servo-based voltage stabilizer that uses a microcontroller to monitor input/output voltage and noise, and intelligently control voltage selection. A MOSFET with PWM enables smooth power delivery and soft-start protection. The use of a 7805 voltage regulator improves measurement accuracy, while noise sensing enhances power quality analysis. With a simple user interface, LCD display, and alert system, the design provides a modern, efficient, and intelligent solution for voltage stabilization.

1.2 HISTORY:-

The concept of voltage stabilization has evolved significantly over time, driven by the increasing demand for reliable and efficient power supply systems. In the early stages, voltage regulation was achieved using simple methods such as manual tap-changing transformers. These systems required human intervention and were not suitable for dynamic conditions. With technological advancements, electromechanical voltage stabilizers were developed. These systems used relays and transformers to switch between different voltage levels based on input conditions. Although they provided automatic regulation, they had limitations such as slow response time, mechanical wear and tear, and limited precision. The introduction of servo-controlled stabilizers marked a major improvement in voltage regulation technology. These stabilizers used servo motors to adjust transformer taps continuously, providing smoother and more accurate voltage correction. However, they were still bulky, expensive, and lacked advanced monitoring capabilities. The development of semiconductor devices led to the emergence of electronic stabilizers, which replaced mechanical components with solid-state devices. These systems offered faster response times and improved reliability but were still limited in terms of intelligence and adaptability. In recent years, the integration of microcontrollers and embedded systems has revolutionized voltage stabilizer design. Modern stabilizers can now monitor multiple parameters, implement complex control algorithms, and provide user interfaces for interaction. They can also incorporate additional features such as fault detection, data logging, and communication capabilities. This project represents a further step in the evolution of voltage stabilizers by integrating real-time monitoring, noise analysis, and self-calibration into a compact embedded system. It combines the advantages of traditional stabilization techniques with modern instrumentation and control methods, resulting in a more advanced and efficient solution.

1.2 PROJECT AIM:-

The primary aim of this project is to design and implement a smart voltage stabilizer that ensures stable and reliable power delivery while incorporating advanced monitoring and control features. The system is intended to improve upon conventional stabilizers by integrating sensing, calibration, and intelligent control into a single platform. The project aims to develop a system capable of selecting appropriate voltage levels automatically or manually, depending on user requirements and operating conditions. It also seeks to implement controlled power delivery using PWM techniques to protect the load and enhance system efficiency. Another important objective is to incorporate real-time monitoring of input

voltage, output voltage, and noise levels. By analyzing these parameters, the system can provide valuable insights into power quality and system performance. The inclusion of a reference voltage for calibration is aimed at improving measurement accuracy and compensating for system variations. This makes the stabilizer more reliable and suitable for applications requiring precise measurements. The project also aims to minimize hardware complexity by using a single potentiometer and a multi-purpose button for user interaction. This demonstrates efficient utilization of resources while maintaining full functionality. Ultimately, the goal is to create an intelligent, flexible, and efficient voltage stabilizer that combines power electronics, embedded systems, and instrumentation techniques. The project serves as a practical demonstration of modern control and monitoring systems and provides a foundation for further advancements in smart power management solutions.

1.4 CHALLENGES:-

1. Maintaining Stable Output

Difficult to keep output voltage constant during input and load variations
Requires fast sensing and control response

2. Accurate Voltage Measurement

ADC of Arduino UNO has limited resolution
Noise affects measurement accuracy
Needs filtering and calibration

3. Noise and Ripple Detection

Power supply noise distorts signals
Hard to differentiate actual noise from measurement error

4. Hardware Resource Limitation

Limited I/O pins
Requires multiplexing and efficient coding

5. Safe Power Delivery

Sudden voltage can damage load
Designing reliable PWM soft-start is challenging

6. User Interface Design

Multiple functions using one potentiometer and button
Needs complex logic for user input handling

7. System Integration

Combining sensing, control, display, and protection is complex
Must ensure reliability and coordination.

CHAPTER 2

2.1 PROCEDURE:-

Step 1: Power Supply Setup

A 12V, 5A SMPS adapter is used as the main power source. The positive terminal of the adapter is distributed to all converter modules, while the negative terminal is connected to a common ground rail shared by all components.

Step 2: Converter Configuration

Four converter modules are used in the system. Buck converter 1 is adjusted to provide a stable 5V supply for the Arduino UNO. Buck converter 2 is set to 5V for powering the SG90 servo motor. Buck converter 3 is configured to provide 12V output, which is connected to the Normally Open (NO) terminal of the relay. A boost converter is configured to provide 24V output, which is connected to the Normally Closed (NC) terminal of the relay.

Step 3: Relay Output Node Setup

The Common (COM) terminal of the relay acts as the main output node. This node supplies power to the load and sensing circuits. The relay switches between 12V and 24V based on control signals from the Arduino.

Step 4: MOSFET Load Control Circuit

A MOSFET is connected in a low-side switching configuration. The positive terminal of the load (fan) is connected to the relay COM output node. The negative terminal of the load is connected to the MOSFET drain. The MOSFET source is connected to ground. The gate is driven using a BJT driver circuit controlled by the Arduino PWM output.

Step 5: MOSFET Indicator LED

An LED with a current limiting resistor is connected between the output node and MOSFET drain. This LED glows when current flows through the MOSFET, indicating actual power delivery to the load.

Step 6: Voltage Sensing Circuits

Voltage divider circuits are used to measure input voltage (V_{in}) and output voltage (V_{out}). Each divider uses a high-value resistor and a low-value resistor to reduce voltage to a safe level for the Arduino Analog inputs. Capacitors are added to filter noise.

Step 7: Noise Sensing Circuit

A noise sensing circuit is implemented using a capacitor connected to the output node and a resistor to ground. The filtered signal is connected to an Analog input pin of the Arduino to measure voltage fluctuations.

Step 8: 7805 Reference Circuit

A 7805 voltage regulator is used to generate a stable reference voltage. Proper input and output capacitors are connected to ensure stability. This reference is used for improving measurement accuracy.

Step 9: User Interface Connections

A potentiometer is connected to an Analog input pin with one end connected to 5V and the other to ground. A push button is connected between a digital input pin and ground using an internal pull-up configuration.

Step 10: Display and Indicators

An LCD display is connected using I2C communication (SDA and SCL pins). Three LEDs are connected to digital output pins to indicate power, stability, and load status. A buzzer is connected through a BJT driver to provide fault alerts.

Step 11: Arduino Integration

All sensors, control circuits, and interface components are connected to the Arduino UNO according to the defined pin mapping. Proper grounding and voltage levels are verified before powering the system.

Step 12: Software Upload

The developed Arduino code is uploaded to the microcontroller using the Arduino IDE. All required libraries are included, and the correct board and port are selected before uploading.

Step 13: Initial Testing

The system is powered on and tested without load. Voltage readings, display output, and control logic are verified. The relay switching and servo movement are tested to ensure proper operation.

Step 14: Load Testing

The fan (load) is connected, and the MOSFET control is tested. PWM operation, soft-start behaviour, and load switching are verified.

Step 15: Fault Testing

Abnormal conditions such as overvoltage and undervoltage are simulated to verify fault detection. The system response, including load disconnection and buzzer activation, is observed.

Step 16: Data Logging Verification

The EEPROM logging feature is tested by checking whether stabilization events are stored and can be retrieved.

Step 17: Final Integration

All components are securely connected, and the system is enclosed properly. Final testing is conducted to ensure reliable and stable operation under different conditions.

This step-by-step procedure ensures accurate implementation and successful operation of the smart voltage stabilizer system.

2.2 DESIGN SPECIFICATIONS:-

1. System Overview

Operates using DC input supply

Provides selectable output: 12V / 24V

Includes monitoring, control, and protection features

2. Power Section

Input from DC adapter uses:

- Buck converter → 12V
- Boost converter → 24V
- Relay selects required voltage

3. Control Unit

Arduino UNO used

Performs:

- Sensing
- Computation
- Control
- Uses ADC for signal processing

4. Sensing System

Voltage dividers for:

- Input voltage (V_{in})
- Output voltage (V_{out})
- Noise sensing using RC circuit
- Ensures safe voltage levels (0–5V)

5. Reference Circuit:- Uses 7805 voltage regulator, Provides stable reference voltage and Improves ADC accuracy through calibration

6. Load Control

MOSFET used as low-side switch

Controlled using PWM

Provides:- Adjustable power and Soft-start protection

7. User Interface

- Potentiometer → multi-function input
- Push button → mode selection/reset
- LCD (I2C) → displays system data
- LEDs → status indication
- Buzzer → fault alert

8. Design Features

Compact and simple hardware

Efficient power control

Reliable and safe operation

Suitable for education and practical use

◆ Electrical Specifications

Parameter	Value
• Input Voltage	12V DC
• Output Voltage	12V / 24V
• Microcontroller Voltage	3.3V
• Servo Supply	5–6V

- Relay Voltage 5V

◆ Measurement Specifications

Parameter	Range
• Vin	0–24V
• Vout	0–24V
• Noise	0–200 mV
• Temperature	0–100°C

◆ Control Specifications

- Servo angle: 0°–180°
- PWM frequency: 5 kHz
- ADC resolution: 12-bit
- Sampling: Average of 10 samples

◆ Functional Specifications

Automatic voltage stabilization
Manual control mode
Soft-start load control

Fault detection:

- Overvoltage
- Undervoltage
- Temperature
- EEPROM data logging
- Dual noise measurement

◆ User Interface Specifications

- 16x2 LCD (I2C)
- Potentiometer control
- Single push button (multi-function)
- LED indicators
- Buzzer alerts

◆ Performance Specifications

- Fast response
- High accuracy (after calibration)
- Stable under load variations
- Reliable data logging

◆ Safety Specifications

- Flyback diode protection
- MOSFET isolation
- Fault lock mechanism
- Controlled load activation

CHAPTER 3

3.1 LIST OF THE COMPONENTS USED IN SERVO VOLTAGE STABILIZER:-

S.No	Component Name	Quantity	Purpose
1.	Arduino UNO Microcontroller	1	Main control and processing unit
2.	16x2 I2C LCD Display	1	Human Machine Interface (HMI)
3.	5V SPDT Relay Module (1-channel)	1	Output Selection(12V\24v)
4.	Toggle Switch / Push Button	1	Multi-function Control
5.	9-12V Buzzer	1	Fault Alert
6.	LEDs	4	Power, Stable, Load, MOSFET Indicators
7.	Breadboard (830 points)	4	Temporary prototyping circuit
8.	Jumper Wires		Wiring and connections
9.	USB Cable Type C	1	Programming via laptop
10.	Arduino IDE Software		Code compilation and upload
11.	12V DC Adapter	1	Power Supply
12.	RESISTORS		
	100k Ω	3	Top Divider (Vin, Vout Buck, Vout Boost)
	10k Ω	4	Bottom Divider + Reference Divider
	1k Ω	4	ADC Protection + BJT base resistors
	10k Ω	1	MOSFET Gate Pull down (Prevents floating)
	220 Ω	1	MOSFET Gate Damping (Reduces Spikes)
	220 Ω	3	Limits LED Current
13.	CAPACITORS		
	1uF (Electrolytic)	1	Noise Measurement-Blocks DC, Passes Ripple
	100uF (Electrolytic)	2	7805 Stability-Input +Output smoothing
	0.1uF (Ceramic)	16	High Frequency Noise Removal
	1000uf 25V/50V (Electrolytic)	3	Power supply stability, Boost converter output
	470uf 25v/50v (Electrolytic)	3	Buck converter output, Boost converter input
	220uf 35V (Electrolytic)	2	Buck converter input(5V), Absorbs ripples
14.	1N4007 Diode		Acts as a fuse
15	BC547 (NPN)Transistor	1	Relay Drivers
	BC547 (NPN)Transistor	1	Buzzer Driver
16.	IRLZ44N9(Logic level N channel)	1	Fan\Load Control(PWM + Soft start)
17.	7805 Linear Regulator	1	Stable Reference Generation
18.	Potentiometer 10k	1	Target Select, Page Scroll, Speed Control
19.	Servo Motor (SG 90S)	1	Range Selection, Stabilization Control
20.	LM2596 Buck Converter	1	5V Output ESP-32,Relay,Display Supply
	LM2596 Buck Converter	1	5 to 5.2V Output, Servo Supply
	LM2596 Buck Converter	1	12V Output, Stabilizer
21.	XL6009 Boost Converter	1	24V Output, Stabilizer

3.2 DESCRIPITON OF EACH COMPONENT:-

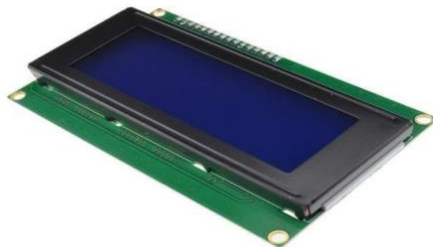
1.Arduino UNO Microcontroller:- Arduino UNO is the central brain of the entire project. It is based on the AT mega 328P microcontroller and is responsible for controlling all devices including LCD, relay, LEDs, buzzer and input switch.

- **Features:-**
- 14 Digital I/O pins(6 PWM pins)
- Analog input pins
- 16 MHz clock speed
- Operates on 5V power
- Built-in EEPROM for storing mission count
- USB programmable using Arduino IDE



2.16x2 LCD Display:- This is a Human Machine Interface (HMI) that shows real time status of the operation. The LCD display is used to display:-

- Voltage values
- Noise levels
- System status
- Graphical indicators



3.Relay Module (5V, 1 channel):- Relays are electromechanical switches which are used to control high-power circuits using low-power signals. In this project, one relay is used to select the output voltage (12V or 24V).It provides electrical isolation and safe switching.



4.Toggle Switch (Push Button):- A single multi-function button is used for target selection, mode switching (Auto/Manual), load activation and system reset.



5.Buzzer :- The buzzer provides audible alerts during fault condition. It enhances system safety by notifying the user.

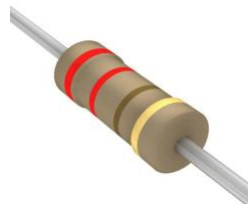


6.LEDs:- Used to indicate the condition of the voltage and system safety. In this project we use three types of LEDs:-

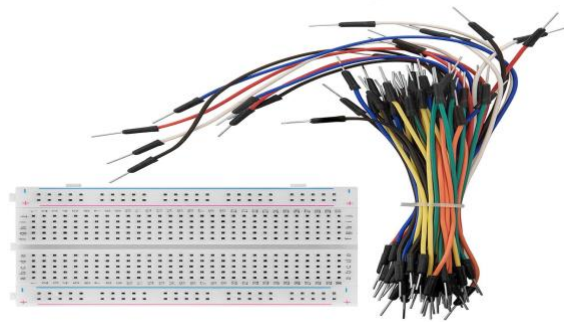
- **Green LED :-** Indicates that the voltage is within the safe range.
- **Yellow LED :-** Indicates slight voltage variation or initial overload condition and acts as a warning signal.
- **Red LED :-** Indicates dangerous voltage condition or overload condition and system may automatically cut off to protect the load.



7.Resistors:- A resistor is a passive electronic component that opposes the flow of electrical current in a circuit and it is used to control current and voltage levels. In this project, we use resistors of different values for multiple roles.



8.Jumper wires and Breadboard:- Used to make connections without soldering. Helps in exploration, modification and safety testing.



9.USB Cable & Arduino IDE:- Used for uploading code and providing power supply.



10.LM2596 Buck Converter:- The LM2596 is step-down switching regulator used to convert a higher input into a lower regulated output voltage. It operates at high efficiency compared to linear regulators. The working principle involves switching the input voltage at high frequency and filtering the output using inductors and capacitor. The feedback pin ensures that the output voltage remains constant by adjusting the duty cycle. In this project, the LM2596 is responsible for the final stabilization of output voltage. This module steps down voltage efficiently. It is used to generate 5V for Arduino and relay and 12V stabilized output. It provides high efficiency and adjustable output.



11.XL6009 Boost Converter:- The XL6009 is a boost converter used to increase the input voltage. It ensures that the system has sufficient voltage headroom for proper operation of the buck converter. The boost converter operates by storing energy in an inductor and releasing it at a higher voltage. This allows the system to achieve output voltages higher than the input. The boost converter increases voltage from 12V to 24V. It is used to provide higher voltage output when required.



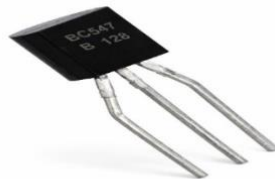
12.IRLZ44N MOSFET:- The MOSFET acts as a high-speed electronic switch. It is used for controlling the load (fan), PWM based operation and soft start operation. It is a logic level MOSFET, suitable for direct microcontroller interfacing (via driver).



13.SG90S Servo Motor:- The servo motor is used for mechanical range selection. It operates based on PWM signals and can rotate to specific angles. Each angle corresponds to a specific voltage range. The servo acts as a bridge between mechanical and electronic control systems. The servo motor adjusts the stabilizer mechanism like controls voltage correction and moves to predefined angles (e.g. 30°,150°).



14.BC547 Transistor:- The BC547 transistor used as a switch. This NPN transistor is used as a driver for relay, MOSFET gate and buzzer. It amplifies the control signal from Arduino.



15.Capacitors:- Capacitor is a passive electrical component that stores electrical energy in form of a electrical field. In this different values of capacitors are used for multiple functions like filtering noise, Blocking DC for ripple measurement, etc.



16.Diodes (IN4007):- Diodes are used for:-

- Reverse polarity protection
- Flyback protection

They ensure safe operation of inductive components like relays and motors.



17.12V 5A SMPS Adapter:- In this project, 12V 5A SMPS Adapter is used to provide a stable and regulated DC power supply to the components, ensuring proper and reliable operation of the system.



18.Potentiometer:- A potentiometer is used in this project to provide variable input for target selection and control operations.



19.Voltage regulators:- A voltage regulator is an electrical component that maintains a constant output voltage. It provides a stable 5V reference. This is used for:-

- ADC calibration
- Improving measurement accuracy



3.3 WORKING OF THE CIRCUIT: -

Overall Working Principle

System works on continuous sensing → decision → control

Input power is given through DC supply

Buck (12V) and Boost (24V) converters generate required voltages

Relay selects required voltage

MOSFET controls power delivery using PWM

Stage 1: Power ON & Initialization

- Arduino UNO initializes system
- LCD displays selection screen
- User selects voltage (12V / 24V)

Stage 2: Voltage Selection

- Servo moves to required position
- Relay switches selected voltage line
- System confirms active voltage

Stage 3: Measurement & Monitoring

Continuously measures:

- Input voltage (V_{in})
- Output voltage (V_{out})
- Noise level
- Voltage dividers + ADC used
- Averaging improves accuracy

Stage 4: Stabilization

System checks difference between target and output

Buck/boost converters internally stabilize voltage

Final stable 12V/24V sent to output

Stage 5: Load Control

Load initially OFF

On button press:

MOSFET turns ON

PWM provides soft start

Smooth power delivery to load

Stage 6: Fault Detection & Protection

Detects:

- Overvoltage
- Undervoltage

Actions:

- MOSFET OFF (disconnect load)
- Buzzer ON

System enters fault lock
Reset required to resume operation

Stage 7: Data Logging

Stores data in EEPROM when system is stable

Saves:

- Output voltage
- Time duration
- Used for performance analysis

Stage 8: User Interface

LCD displays:

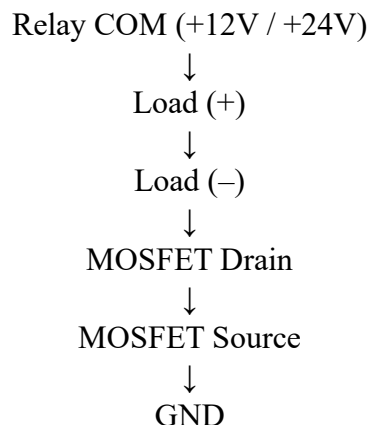
- Voltage
- Noise
- Mode
- LEDs → status indication
- Buzzer → alerts
- Potentiometer + button → user control

MOSFET POWER & SIGNAL FLOW (IRLZ44N)

1. BASIC CONFIGURATION:- The MOSFET (IRLZ44N) is used in a low-side switching configuration to control power delivery to the load (fan). The drain terminal of the MOSFET is connected to the negative terminal of the load, while the source terminal is connected to ground. The positive terminal of the load is connected to the system output node (relay COM). When the MOSFET is turned ON, it provides a conductive path from drain to source, allowing current to flow through the load. When it is turned OFF, this path is interrupted, and the load is disconnected.

2. POWER FLOW

MOSFET ON STATE



Current flows → Load ON
MOSFET acts like a closed switch

MOSFET OFF STATE

Drain–Source path open

→ No current flow

→ Load OFF

3. GATE CONTROL (SIGNAL FLOW):- The gate terminal of the MOSFET is controlled by the Arduino through a BJT driver circuit. The Arduino outputs a PWM signal, which is amplified by the BJT to provide sufficient gate drive. When a voltage greater than the threshold voltage (typically 2–4V for IRLZ44N) is applied to the gate, the MOSFET turns ON. When the gate voltage is removed or reduced to zero, the MOSFET turns OFF.

The gate behaves like a capacitor, requiring charging and discharging for switching operations.

4. GATE DAMPING (VERY IMPORTANT):- A gate resistor (typically 100Ω–220Ω) is used between the driver and the MOSFET gate. This resistor is known as the gate damping resistor. It limits the inrush current during charging of the gate capacitance and prevents high-frequency oscillations. Without this resistor, the MOSFET may experience ringing and unstable switching, leading to electromagnetic interference (EMI) and potential damage. Additionally, a pull-down resistor (10kΩ) is connected between the gate and ground to ensure that the MOSFET remains OFF when no signal is applied.

5. INRUSH CURRENT LIMITING:- When the MOSFET turns ON, the load (fan) may draw a high initial current known as inrush current. This can stress the MOSFET and power supply. To control this, PWM-based soft-start is implemented. Instead of applying full voltage instantly, the PWM duty cycle is gradually increased. This results in a smooth rise in current, reducing stress on components and improving system reliability. The gate resistor also helps in limiting the rate of voltage change (dv/dt), indirectly reducing inrush effects.

6. Rds(on) AND POWER LOSS

The IRLZ44N MOSFET has a low drain-to-source resistance (Rds(on)) when fully turned ON. This ensures minimal voltage drop across the MOSFET and reduces power loss.

Power dissipation in the MOSFET is given by:

$$P = I^2 \times R_{ds(on)}$$

A low Rds(on) value ensures efficient operation and reduced heating. However, proper heat dissipation should still be considered for higher load currents.

7. BODY DIODE (IMPORTANT):- The MOSFET contains an internal body diode between the drain and source terminals. This diode provides a path for reverse current, especially in inductive loads such as motors or fans. For additional protection, an external flyback diode is often connected across the load to suppress voltage spikes generated during switching.

8. MOSFET LED INDICATOR:- An LED is connected between the output node and the MOSFET drain. When the MOSFET conducts, current flows through the LED, causing it to glow. This provides a visual indication that power is being delivered to the load. This LED confirms actual current flow rather than just control signal presence, making it a reliable indicator of system operation.

9. SIGNAL + POWER SEPARATION:- The MOSFET separates the low-power control signal (from Arduino) from the high-power load circuit. The gate requires only a small control current, while the drain-source path handles the load current. This allows efficient switching of high-power loads using low-power control signals, which is a key advantage in embedded systems.

BJT DRIVER EXPLANATION

PURPOSE OF BJT DRIVER:- The BJT driver circuit is used to provide sufficient current to drive the gate of the MOSFET efficiently. Since the Arduino output pins have limited current capability, the BJT acts as an intermediate amplifier between the microcontroller and the MOSFET. The BJT ensures faster switching of the MOSFET by supplying adequate base and gate current, improving overall system performance.

WORKING PRINCIPLE

When the Arduino outputs a HIGH signal or PWM signal, current flows into the base of the BJT through the base resistor ($1k\Omega$). This forward-biases the base-emitter junction, turning the BJT ON. When the BJT turns ON, it allows current to flow from collector to emitter, effectively pulling the MOSFET gate towards ground or driving it depending on configuration. This rapid charging and discharging of the MOSFET gate improves switching speed. When the Arduino output is LOW, the BJT turns OFF, and the MOSFET gate is pulled down to ground through the pull-down resistor, ensuring the MOSFET remains OFF.

Base Resistor ($1k\Omega$) Limits base current and protects both Arduino and BJT from excessive current, Gate Resistor (220Ω) Controls the charging current of the MOSFET gate and prevents oscillations, Pull-down Resistor ($10k\Omega$) Ensures the MOSFET remains OFF when no signal is applied, preventing false triggering and Gate Capacitor ($0.1\mu F$) Provides noise filtering and stabilizes the gate voltage, reducing switching spikes.

The BJT driver improves switching performance, reduces load on the Arduino, enables faster MOSFET operation, and enhances overall system reliability.

1. MAIN POWER SOURCE

12V SMPS Adapter

Supplies power to:

Buck converters

Boost converter

2. CONVERTER STAGE

The converters modify the input voltage into required levels. Buck converter 1 generates 5V for the Arduino, buck converter 2 generates 5V for the servo motor, and buck converter 3 generates 12V for one range of operation. The boost converter steps up the voltage to 24V for the higher range.

3. RELAY SWITCHING

Buck 3 (12V) → Relay NO

Boost (24V) → Relay NC

Relay COM → Output Node

Arduino controls relay

Selects 12V or 24V

4. OUTPUT NODE

The relay common terminal acts as the system output node. This node distributes power to the load and sensing circuits. It is the central point where the selected voltage is available.

5. MOSFET CONTROL STAGE

Output Node (+) → Load (+)
Load (−) → MOSFET Drain
MOSFET Source → GND
MOSFET controls current flow

6. LOAD OPERATION

When the MOSFET is turned ON, current flows from the output node through the load and into the MOSFET, completing the circuit to ground. When the MOSFET is OFF, the circuit is open, and the load is disconnected.

LOW-SIDE SWITCHING

MOSFET placed at ground side
Easy control
Efficient

CURRENT PATH

+V → Load → MOSFET → GND

VOLTAGE DROP

Minimal due to low $R_{ds(on)}$

HEAT GENERATION

$$P = I^2 \times R_{ds(on)}$$

SOFT START

PWM gradually increases
Reduces inrush current

FLYBACK PROTECTION

For fan (inductive load):
Diode across load recommended

FINAL SUMMARY

BJT DRIVER:- Amplifies control signal, Drives MOSFET gate and Improves switching

POWER STAGE

Converts voltage

Selects range

Controls load

Ensures safe power delivery

MULTI-FUNCTION POTENTIOMETER:- A single potentiometer is used as a multi-functional input device in the system. Its function changes dynamically based on the current operating mode. In manual load mode, the potentiometer controls the PWM duty cycle of the MOSFET, thereby adjusting the power delivered to the load. In menu or selection modes, the potentiometer is used to scroll through options such as display pages or mode selection. This multiplexing approach reduces hardware complexity while maintaining full control functionality.

◆ POT FUNCTION TABLE

Mode	POT Function
-----	-----
Manual Load Mode	Controls PWM (fan speed)
Display Mode	Scrolls pages
Mode Selection	Selects Auto/Manual

MULTI-FUNCTION PUSH BUTTON:- The push button is used as a multi-purpose control input. It performs different actions depending on the system state and duration of press. A short press is used for toggling load ON/OFF or confirming selections. A long press is used for system reset, especially during fault conditions. This design enables multiple functionalities using a single input device.

◆ BUTTON FUNCTION TABLE

Condition	Action
-----	-----
Short Press	Toggle Load ON/OFF
Short Press (Menu)	Confirm Selection
Fault Condition	Reset system
Long Press	Full system reset

◆ SYSTEM STATES

1. POWER ON
2. MODE SELECTION
3. NORMAL OPERATION
4. LOAD CONTROL
5. STABLE STATE
6. FAULT STATE
7. RESET STATE

◆ **STABILITY LOGGING:-** If stability is more than 95% then event is stored.

MATHEMATICAL & LOGICAL CALCULATIONS

1. VOLTAGE DIVIDER DERIVATION

The voltage divider is used to scale down high voltages (V_{in} and V_{out}) to a safe level for the Arduino ADC.

For a voltage divider with resistors R_1 and R_2 :

$$V_{adc} = V_{in} \times (R_2 / (R_1 + R_2))$$

In this project:

$$R_1 = 100k\Omega$$

$$R_2 = 10k\Omega$$

Therefore:

$$V_{adc} = V_{in} \times (10k / (100k + 10k))$$

$$V_{adc} = V_{in} \times (10 / 110)$$

$$V_{adc} \approx V_{in} \times 0.0909$$

To calculate actual voltage:

$$V_{in} = V_{adc} \times (110 / 10)$$

$$V_{in} = V_{adc} \times 11$$

This scaling factor is used in the code to convert ADC readings into real voltage values.

2. ADC CONVERSION

The Arduino UNO uses a 10-bit ADC, which means it converts Analog voltage (0–5V) into digital values (0–1023).

$$\text{ADC value} = (V_{in} / 5) \times 1023$$

To convert back:

$$V_{in} = (\text{ADC value} \times 5) / 1023$$

Combining with divider scaling:

$$\text{Actual Voltage} = ((\text{ADC} \times 5) / 1023) \times 11$$

3. PWM DUTY CYCLE

PWM is used to control power delivered to the load.

$$\text{Duty Cycle (\%)} = (T_{on} / \text{Total Time}) \times 100$$

In Arduino:

$$\text{PWM range} = 0 \text{ to } 255$$

$$\text{Duty Cycle} = (\text{PWM value} / 255) \times 100$$

Example:

$$\text{PWM} = 128 \rightarrow \text{Duty} \approx 50\%$$

This allows smooth control of load power.

4. MOSFET POWER DISSIPATION

The power loss in MOSFET is calculated using:

$$P = I^2 \times R_{ds(on)}$$

For IRLZ44N:

$$R_{ds(on)} \approx 0.022\Omega$$

If load current = 0.15A:

$$P = (0.15)^2 \times 0.022$$

$$P \approx 0.0005W$$

This shows very low power loss, indicating high efficiency.

5. IN-RUSH CURRENT CONTROL (PWM)

Inrush current occurs when the load is suddenly powered.

By gradually increasing PWM:

Current $\propto$ Duty Cycle

Initial low duty cycle $\rightarrow$ low current

Gradual increase $\rightarrow$ smooth rise

This prevents sudden surge and protects components.

6. STABILITY CALCULATION

Stability is calculated based on deviation from target voltage.

$$\text{Error} = |\text{Target Voltage} - V_{\text{out}}|$$

$$\text{Stability} = 100 - (\text{Error} \times K)$$

Where K is a scaling constant (e.g., 5)

Example:

$$\text{Target} = 12V$$

$$V_{\text{out}} = 11.8V$$

$$\text{Error} = 0.2$$

$$\text{Stability} = 100 - (0.2 \times 5)$$

$$\text{Stability} = 99\%$$

Higher value indicates better stability.

7. NOISE CALCULATION

Noise is measured by sampling multiple ADC values.

$$\text{Noise} = (\text{Max value} - \text{Min value}) \times 1000$$

This gives noise in millivolts (mV).

This method detects ripple and fluctuations in output voltage.

8. FAULT DETECTION LOGIC

Fault conditions are defined using threshold limits.

Overvoltage condition:

$$V_{\text{out}} > 26V$$

Undervoltage condition:

$$V_{\text{out}} < 10V$$

Logical condition:

$$\text{Fault} = (V_{\text{out}} > \text{Upper Limit}) \text{ OR } (V_{\text{out}} < \text{Lower Limit})$$

If fault is true, protective actions are triggered.

9. EEPROM DATA STORAGE LOGIC

EEPROM is used to store event data.

Stored parameters:

Voltage $\rightarrow$ integer form

Time $\rightarrow$ seconds (millis / 1000)

Low byte:

t & 0xFF

High byte:

$(t \gg 8) \& 0xFF$

This splits time into two bytes for storage.

10. GATE RESISTOR CALCULATION

The gate resistor limits charging current of MOSFET gate.

$$I_g = V / R$$

For Arduino output 5V and $R = 220\Omega$:

$$I_g = 5 / 220 \approx 22\text{mA}$$

This protects the driving circuit and prevents oscillations.

11. VOLTAGE REFERENCE DIVIDER (7805)

To obtain 2.5V from 5V:

$$R_1 = R_2 = 10\text{k}\Omega$$

$$V_{\text{ref}} = 5 \times (10\text{k} / (10\text{k} + 10\text{k}))$$

$$V_{\text{ref}} = 2.5\text{V}$$

This provides midpoint reference for ADC calibration.

12. POWER FLOW EFFICIENCY

Efficiency depends on losses in converters and MOSFET.

$$\text{Efficiency} \approx (\text{Output Power} / \text{Input Power}) \times 100$$

Since MOSFET losses are very low and converters are efficient, overall system efficiency is high.

FINAL SUMMARY:- PWM SOFT START & RAMP CONTROL (GENERAL LOAD)

The MOSFET-based PWM soft-start and ramp control technique is implemented to regulate power delivery to the load in a controlled and efficient manner. Instead of applying full voltage instantaneously, the system gradually increases the PWM duty cycle from zero to the required value. This gradual increase in duty cycle results in a smooth rise in output voltage and current, thereby reducing inrush current and preventing sudden stress on the load and power components. Although a DC fan is used as a demonstration load in this project, the same control technique can be applied to a wide range of electrical loads. These include DC motors, LED lighting systems, heating elements, battery charging circuits, and other electronic devices. The PWM ramp ensures that the system behaves as a controlled power delivery unit rather than a simple ON/OFF switch. This improves reliability, enhances efficiency, and extends the lifespan of connected loads. The MOSFET operates as a high-speed electronic switch controlled by the PWM signal. By varying the duty cycle, the average power delivered to the load is adjusted. This allows precise control over load operation regardless of the type of load connected. In applications involving inductive loads such as motors, the soft-start feature is particularly important as it minimizes mechanical shock and electrical transients. For resistive loads such as heaters, it prevents sudden current surges and thermal stress. Thus, the PWM soft-start and ramp control technique makes the system versatile and suitable for multiple practical and industrial applications beyond the demonstration setup.

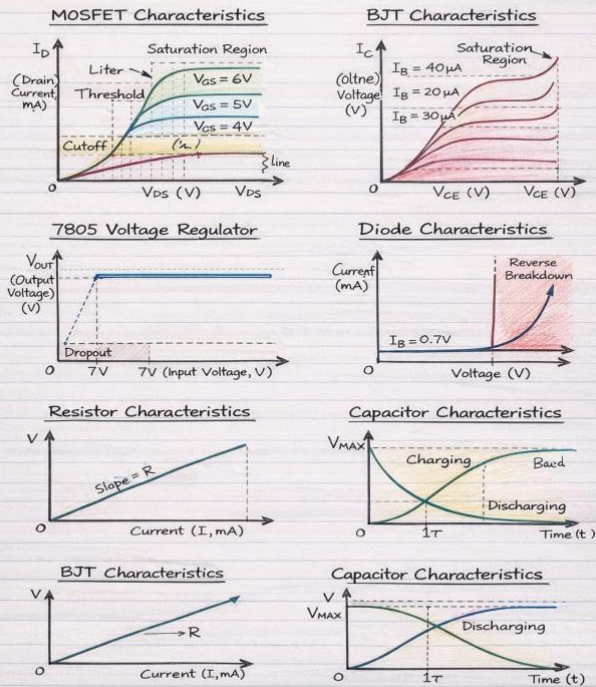
The average output voltage applied to the load is proportional to the PWM duty cycle and can be expressed as:

$$V_{\text{avg}} = D \times V_{\text{supply}}$$

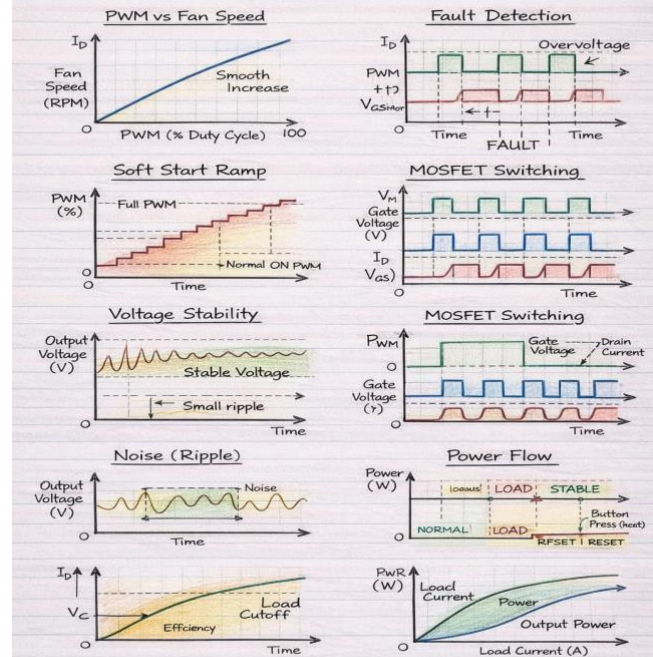
where D is the duty cycle (0 to 1) and V_{supply} is the selected voltage (12V or 24V).

By gradually increasing the duty cycle, the system ensures a controlled increase in output voltage and current, achieving a soft-start effect.

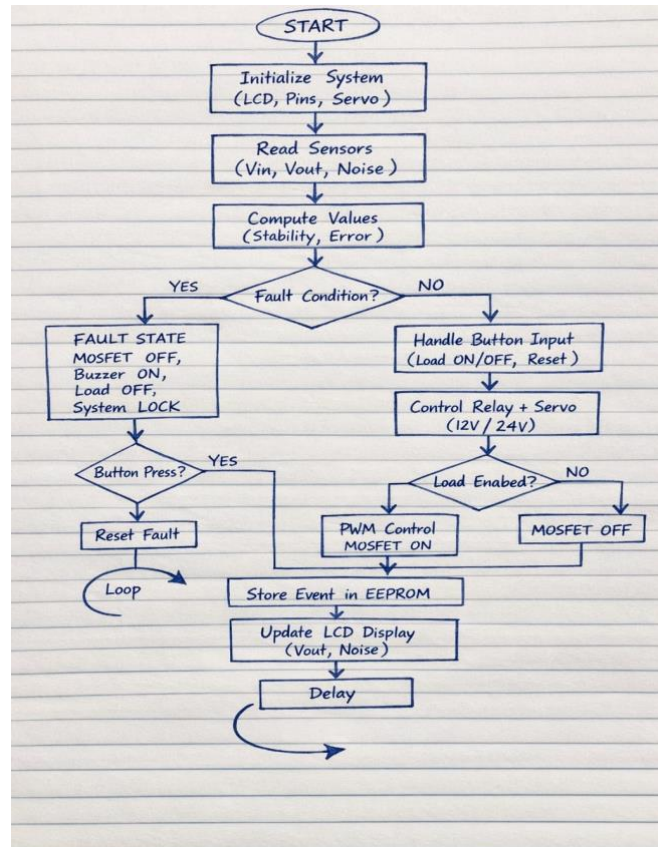
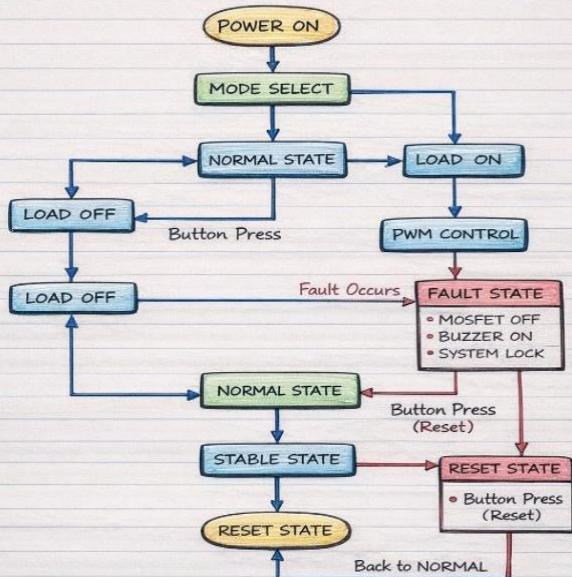
V-I Characteristics



Graphs



State Diagram



CHAPTER 4

4.1 SOURCE CODE

1. Purpose of Code

Performs:

- Sensing
- Control
- Protection
- Data logging
- Implemented using Arduino UNO

2. Libraries Used

- Wire → I2C communication
- LiquidCrystal_I2C → LCD display
- EEPROM → data storage
- Servo → servo motor control

3. Pin Configuration

Analog Pins (A0–A3):

- Input voltage (Vin), Output voltage (Vout), Noise sensing and Potentiometer input

Digital Pins:

- Relay control, MOSFET control, Servo motor, Buzzer, LEDs and Push button

4. ADC & Voltage Calculation

- Reads Analog values from sensors
- Converts ADC values into actual voltage
- Uses scaling factor (voltage divider ratio)

5. Noise Measurement

Takes multiple samples and calculates:

- Max value
- Min value
- Difference → represents noise/ripple

6. Sensor Update Function

Continuously updates:

- Input voltage
- Output voltage
- Noise level

7. Stability Calculation

Compares:

- Target voltage
- Measured output

- Controls LED to indicate stability

8. Fault Detection System

Detects:

- Overvoltage
- Undervoltage

Actions:

- Turns OFF MOSFET (disconnect load)
- Activates buzzer
- Locks system until reset

9. Control System

MOSFET:

- Controls load using PWM
- Duty cycle adjusted using potentiometer
- Relay + Servo:
- Selects voltage range (12V / 24V)

10. User Interaction

Push button:

- Toggle load ON/OFF
- Reset system during fault
- Potentiometer:
- Controls output power

11. Data Logging (EEPROM)

Stores:

- Output voltage
- Time when system becomes stable
- Used for analysis

12. LCD Display

Displays:

- Output voltage
- Noise level
- Updates continuously in real-time

13. Program Structure

Setup Function and initializes:

- Pins
- LCD
- Servo
- System components

4.2 CODE INSTALLATION

The code for the project is developed and uploaded using the Arduino Integrated Development Environment (IDE). The following steps are followed for installation and execution:

1. Install the Arduino IDE on the computer from the official website.
2. Connect the Arduino UNO board to the computer using a USB cable.
3. Open the Arduino IDE and select the correct board type (Arduino UNO) from the Tools menu.
4. Select the appropriate COM port corresponding to the connected Arduino board.
5. Include the required libraries such as Wire and LiquidCrystal_I2C. These can be installed using the Library Manager in the IDE.
6. Copy and paste the project source code into the Arduino IDE editor.
7. Verify the code to check for syntax errors using the compile option.
8. Upload the code to the Arduino board by clicking the upload button.
9. Once the code is uploaded successfully, the system will start executing automatically.

After uploading the code, the hardware connections should be verified, and the system should be powered using the external supply. The LCD display should initialize and start showing system parameters. The push button and potentiometer can then be used to interact with the system. Proper precautions should be taken to ensure correct wiring and avoid short circuits. All voltage levels should be verified before connecting the Arduino inputs. Once verified, the system is ready for operation and testing.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <EEPROM.h>
#include <Servo.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo servo;
// ----- PIN MAP -----
#define VIN_PIN    A0
#define VOUT_PIN   A1
#define POT_PIN    A2
#define NOISE_PIN  A3
#define RELAY_PIN  8
#define MOSFET_PIN 9
#define SERVO_PIN  10
#define BUTTON_PIN 2
#define BUZZER     4
#define LED_POWER  5
#define LED_STABLE 6
#define LED_LOAD   7
// ----- VARIABLES -----
float Vin = 0, Vout = 0, Noise = 0;
float stability = 0;
int targetVoltage = 12;
bool loadEnabled = false;
bool fault = false;
```

```

bool eventStored = false;
int pwmValue = 0;
// ----- ADC READ -----
float readADC(int pin)
{
    return analogRead(pin) * (5.0 / 1023.0);
}
// ----- NOISE -----
float measureNoise(int pin)
{
    float maxV = 0, minV = 5;
    for(int i = 0; i < 20; i++)
    {
        float v = readADC(pin);
        if(v > maxV) maxV = v;
        if(v < minV) minV = v;
    }
    return (maxV - minV) * 1000;
}
// ----- SENSOR -----
void readSensors()
{
    Vin = readADC(VIN_PIN) * 11; // divider scale
    Vout = readADC(VOUT_PIN) * 11;
    Noise = measureNoise(NOISE_PIN);
}
// ----- COMPUTE -----
void computeValues()
{
    float error = abs(targetVoltage - Vout);
    stability = 100 - (error * 5);
    if(stability > 95)
        digitalWrite(LED_STABLE, HIGH);
    else
        digitalWrite(LED_STABLE, LOW);
}
// ----- EEPROM LOG -----
void storeEvent()
{
    int v = (int)Vout;
    int t = millis() / 1000;

    EEPROM.write(0, v);
    EEPROM.write(1, t & 0xFF);
    EEPROM.write(2, (t >> 8) & 0xFF);
    eventStored = true;
}

```

```

// ----- BUTTON -----
void handleButton()
{
    static bool lastState = HIGH;
    bool current = digitalRead(BUTTON_PIN);
    if(lastState == HIGH && current == LOW)
    {
        delay(200);
        if(fault)
        {
            fault = false;
            digitalWrite(BUZZER, LOW);
            return;
        }
        loadEnabled = !loadEnabled;
    }
    lastState = current;
}

// ----- FAULT DETECTION -----
void checkFault()
{
    if(Vout > 26 || Vout < 10)
    {
        fault = true;
    }
}

// ----- CONTROL -----
void controlSystem()
{
    if(fault)
    {
        analogWrite(MOSFET_PIN, 0);
        digitalWrite(BUZZER, HIGH);
        digitalWrite(LED_LOAD, LOW);
        return;
    }
    digitalWrite(BUZZER, LOW);
    if(loadEnabled)
    {
        pwmValue = map(analogRead(POT_PIN), 0, 1023, 80, 255);
        analogWrite(MOSFET_PIN, pwmValue);
        digitalWrite(LED_LOAD, HIGH);
    }
    else
    {
        analogWrite(MOSFET_PIN, 0);
        digitalWrite(LED_LOAD, LOW);
    }
}

```

```

}
// ----- RELAY + SERVO -----
void controlRelay()
{
  if(targetVoltage == 12)
  {
    digitalWrite(RELAY_PIN, HIGH);
    servo.write(40);
  }
  else
  {
    digitalWrite(RELAY_PIN, LOW);
    servo.write(140);
  }
}
// ----- DISPLAY -----
void displayData()
{
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("V:");
  lcd.print(Vout,1);
  lcd.setCursor(0,1);
  lcd.print("N:");
  lcd.print(Noise,0);
}
// ----- SETUP -----
void setup()
{
  lcd.init();
  lcd.backlight();
  pinMode(RELAY_PIN, OUTPUT);
  pinMode(MOSFET_PIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(BUZZER, OUTPUT);
  pinMode(LED_POWER, OUTPUT);
  pinMode(LED_STABLE, OUTPUT);
  pinMode(LED_LOAD, OUTPUT);
  digitalWrite(LED_POWER, HIGH);
  servo.attach(SERVO_PIN);
}
// ----- LOOP -----
void loop()
{
  readSensors();
  computeValues();
  handleButton();
}

```

```

checkFault();
controlRelay();
// ----- DATA LOGGING -----
if(stability > 95 && !eventStored)
    storeEvent();
if(stability < 90)
    eventStored = false;
controlSystem();
displayData();
delay(200);
}

```

4.3 CODE EXPLANATION

The source code of the smart servo-based voltage stabilizer is designed to perform sensing, control, protection, and data logging operations using the Arduino UNO microcontroller.

The program begins with the inclusion of required libraries such as Wire for I2C communication, LiquidCrystal_I2C for LCD display, EEPROM for data storage, and Servo for controlling the servo motor. These libraries enable communication with external devices and implementation of advanced features. Pin definitions are declared to map the hardware connections with the microcontroller. Analog pins A0 to A3 are used for sensing input voltage, output voltage, noise, and potentiometer input. Digital pins are assigned for controlling the relay, MOSFET, servo motor, buzzer, LEDs, and push button.

The Analog-to-digital conversion function reads sensor values from the voltage divider circuits and converts them into actual voltage values. A scaling factor is applied to compensate for the voltage divider ratio. Noise measurement is performed by sampling multiple readings and calculating the difference between maximum and minimum values, representing signal fluctuations.

The sensor reading function continuously updates the values of input voltage, output voltage, and noise. These values are used in further computations to determine system performance.

The computation block calculates the stability of the system based on the difference between the target voltage and the measured output voltage. A stability indicator LED is controlled based on this calculation to provide visual feedback. A fault detection mechanism is implemented to monitor abnormal conditions. If the output voltage exceeds predefined upper or lower limits, the system enters a fault state. In this condition, the MOSFET is turned off to disconnect the load, the buzzer is activated, and the system remains locked until reset by the user. The control system manages the operation of the MOSFET and relay. The MOSFET controls power delivery to the load using PWM signals. The duty cycle of PWM is adjusted using the potentiometer input, allowing manual control of power. The relay and servo motor work together to select the required voltage range (12V or 24V).

User interaction is handled through a push button, which toggles load operation and resets the system in case of a fault. The potentiometer is used as a multi-purpose input for controlling output power.

The program also includes an EEPROM-based data logging feature. When the system reaches a stable condition, the output voltage and elapsed time are stored in memory. This allows retrieval of important system events for analysis. The LCD display is used to show real-time system parameters such as output voltage and noise level. The display is updated continuously to provide clear information to the user.

The setup function initializes all hardware components, including pin configurations, LCD initialization, and servo attachment. The loop function continuously executes the main logic, including sensor reading, computation, fault detection, control actions, data logging, and display updates. Overall, the code integrates multiple functionalities into a single program, ensuring efficient and reliable operation of the voltage stabilizer system.

4.4 ADVANTAGES OF SOFTWARE DESIGN

1. Real-Time Monitoring & Control

- Continuously reads sensor data
- Updates system instantly
- Responds quickly to voltage changes

2. Fault Detection & Protection

Detects:

- Overvoltage
- Undervoltage
- Takes action:
- Disconnects load
- Activates buzzer/alerts

3. Efficient PWM Control

- Uses PWM for power control
- Provides smooth output
- Enables soft-start → protects devices

4. Automatic & Manual Modes

- Automatic mode → system works independently
- Manual mode → user controls operation
- Provides flexibility

5. Data Logging (EEPROM)

- Stores important system events
- Helps in analysis and performance evaluation

6. Modular Code Structure

- Code divided into functions
- Easy to understand, debug, and modify
- Supports future upgrades

7. Reduced Hardware Complexity

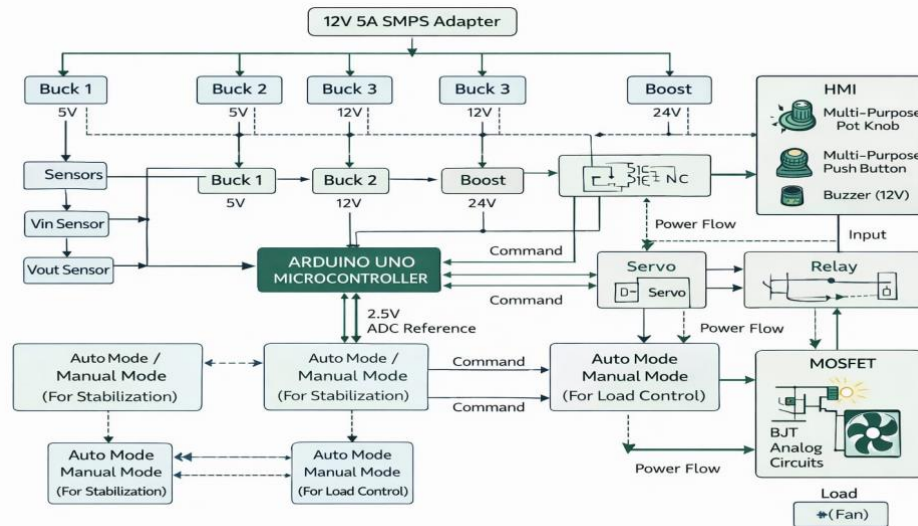
- One potentiometer + one button for multiple functions
- Achieved using smart programming
- Reduces cost and components

8. User-Friendly Interface

- LCD displays system parameters clearly
- Improves interaction and usability

CHAPTER 5

5.1 BLOCK DIAGRAM



The block diagram of the smart servo-based voltage stabilizer represents the overall functional structure of the system. It shows how different components are interconnected and how data and power flow between them. The block diagram helps in understanding the working of the system at a high level without focusing on detailed circuit connections. It divides the system into major functional blocks such as power supply, sensing unit, control unit, user interface, protection system, and data logging unit. Each block performs a specific function, and all blocks work together to achieve stable voltage regulation, load control, monitoring, and protection. The diagram provides a clear visualization of how input signals are processed and how control actions are generated in response.

1. POWER SUPPLY BLOCK:- This block consists of a 12V SMPS adapter and multiple converter modules. Buck converters are used to generate regulated 5V and 12V supplies, while a boost converter generates 24V. These outputs provide power to the control system, servo motor, and load.

2. VOLTAGE SELECTION BLOCK:- This block includes the relay and servo mechanism. The servo positions the selection mechanism, and the relay switches between 12V and 24V outputs. The selected voltage is passed to the system output node.

3. OUTPUT NODE (SYSTEM OUTPUT):- The output node is the central point where the selected voltage is available. This node supplies power to the load, sensing circuits, and monitoring units.

4. SENSING BLOCK:- The sensing block includes Vin sensor, Vout sensor, and noise sensor circuits. These sensors measure input voltage, output voltage, and signal fluctuations. The sensed values are fed to the microcontroller for processing.

5. CONTROL UNIT (ARDUINO):- The Arduino UNO acts as the central processing unit of the system. It reads sensor data, processes information, makes decisions, and controls the relay, MOSFET, servo, and other components.

6. LOAD CONTROL BLOCK:- This block consists of a MOSFET and its driver circuit. The MOSFET controls the power supplied to the load using PWM signals, enabling smooth and controlled operation.

7. USER INTERFACE BLOCK:- The user interface includes a potentiometer, push button, LCD display, LEDs, and a buzzer. These components allow user interaction, system control, and provide visual and audible feedback.

8. FAULT DETECTION & PROTECTION BLOCK:- This block detects abnormal conditions such as overvoltage and undervoltage. It initiates protective actions such as disconnecting the load, activating the buzzer, and locking the system.

9. DATA LOGGING BLOCK:- This block uses EEPROM memory to store important system events such as stabilization. It enables analysis of system performance and enhances monitoring capabilities.

10. REFERENCE BLOCK (7805):- The 7805 voltage regulator provides a stable reference voltage for the Arduino ADC. This improves measurement accuracy and ensures reliable sensing.

5.2 BLOCK DIAGRAM DESCRIPTION

The block diagram of the smart servo-based voltage stabilizer illustrates the functional relationship between different subsystems involved in power conversion, sensing, control, protection, and data logging. The system operates by continuously monitoring voltage conditions and taking appropriate actions to maintain stable output. The operation begins with the power supply block, which consists of a 12V SMPS adapter. This input is distributed to multiple converter modules. Buck converter 1 generates a regulated 5V supply for the Arduino UNO, while buck converter 2 provides 5V for the servo motor. Buck converter 3 produces 12V output, and a boost converter generates 24V output. These voltage levels are used for range selection and load supply. The voltage selection block consists of a relay and a servo mechanism. The servo is controlled by the Arduino to position the selection mechanism, and the relay switches between the 12V and 24V outputs. The selected voltage is then available at the relay common terminal, which acts as the system output node. The output node is a central point in the system from which power is distributed to the load and sensing circuits. It ensures that only the selected voltage (either 12V or 24V) is supplied to the load.

The sensing block includes three sensors: input voltage sensor (V_{in}), output voltage sensor (V_{out}), and noise sensor. These sensors use voltage divider and filtering circuits to convert high voltage signals into safe levels suitable for the Arduino Analog inputs. The noise sensor detects fluctuations and ripple in the output signal. The control unit is implemented using an Arduino UNO microcontroller. It continuously reads sensor data, processes it, and makes decisions based on predefined logic. It controls the relay for voltage selection, the servo for positioning, and the MOSFET for load control.

The load control block consists of a MOSFET and a BJT driver circuit. The MOSFET operates as a low-side switch and controls power delivery to the load using PWM signals. This allows smooth operation and prevents sudden current surges.

The user interface block includes a potentiometer, push button, LCD display, LEDs, and a buzzer. The potentiometer is used as a multi-purpose input for controlling system parameters, while the push button is used for load control and fault reset. The LCD displays real-time values such as output voltage and noise, while LEDs and buzzer provide status indications. The fault detection and protection block continuously checks for abnormal conditions such as overvoltage and undervoltage. When a fault is

detected, the system disconnects the load by turning off the MOSFET, activates the buzzer, and enters a fault lock state to prevent further operation until reset. The data logging block uses EEPROM memory to store important system events such as stabilization. This allows retrieval of previous data for analysis and monitoring of system performance.

The reference block includes a 7805 voltage regulator, which provides a stable reference voltage for the Arduino ADC. This improves the accuracy and reliability of voltage measurements.

Overall, the block diagram represents a closed-loop system where sensing, processing, and control are integrated to achieve stable and safe operation. The combination of monitoring, control, protection, and logging makes the system efficient and reliable for practical applications.

5.3 SENSOR SYSTEM

1. INPUT VOLTAGE SENSOR (Vin SENSOR):- The input voltage sensor is used to measure the voltage supplied by the adapter. Since the Arduino Analog input pins can handle a maximum of 5V, a voltage divider circuit is used to scale down the input voltage to a safe level. The voltage divider consists of a high-value resistor (100kΩ) connected in series with a low-value resistor (10kΩ). The input voltage is applied across the divider, and the reduced voltage is taken from the junction of the two resistors. This scaled voltage is then fed to the Arduino Analog input pin.

The division ratio is given by:

$$V_{out} = V_{in} \times (10k / (100k + 10k))$$

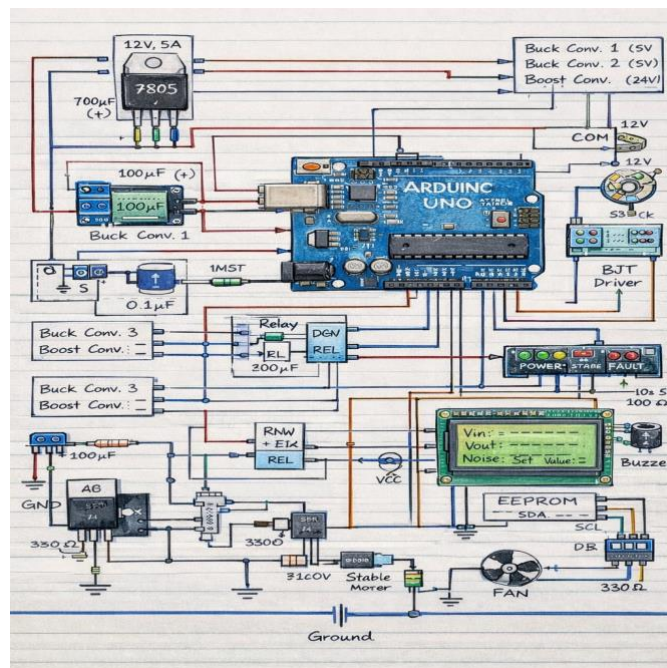
This ensures that even if the input voltage reaches higher values, the Arduino receives a proportionally reduced voltage within its safe operating range. A capacitor (0.1μF) is connected from the sensing node to ground to filter high-frequency noise and provide a stable DC signal. Power flow in this sensor is minimal, as it is a high-impedance circuit designed only for measurement purposes. This reduces power loss and improves efficiency.

2. OUTPUT VOLTAGE SENSOR (Vout SENSOR):- The output voltage sensor measures the voltage available at the system output node, which is the relay common terminal. Similar to the input voltage sensor, a voltage divider is used to scale down the output voltage. The same resistor combination (100kΩ and 10kΩ) is used to maintain consistency in measurement. The scaled voltage is connected to the Arduino Analog input pin. A capacitor (0.1μF) is added to filter noise and stabilize the measurement. This is particularly important because the output voltage may contain switching noise due to relay operation and load variations. The sensor draws very little current due to the use of high-value resistors, ensuring that it does not affect the actual output voltage supplied to the load. This sensor plays a crucial role in feedback control, as the Arduino uses this value to determine system stability and detect abnormal conditions such as overvoltage and undervoltage.

3. NOISE SENSOR (OUTPUT RIPPLE SENSOR):- The noise sensor is designed to detect fluctuations and ripple present in the output voltage. It is implemented using a coupling capacitor and a resistor to ground. A capacitor (typically 1μF) is connected between the output node and the sensing node. This capacitor blocks DC components and allows only AC variations (noise) to pass through. The sensing node is connected to ground through a resistor (100kΩ), forming a high-pass filter. The Arduino reads the voltage at this node and calculates noise by sampling multiple readings and determining the difference between maximum and minimum values.

The capacitor polarity is important. The positive terminal is connected to the output node, and the negative terminal is connected to the sensing node. This circuit does not draw significant power, as it is used only for signal extraction. It provides valuable information about system quality and stability by indicating the level of voltage ripple.

4. REFERENCE VOLTAGE SENSOR (7805 BASED):- A 7805 voltage regulator is used to generate a stable 5V reference for improving ADC accuracy. The input of the 7805 is connected to a higher voltage source (typically 12V), and the output provides a regulated 5V supply. To obtain a reference value suitable for the Arduino ADC, a voltage divider is used to scale the 5V output to approximately 2.5V using two equal resistors (10k Ω each). Capacitors (100 μ F and 0.1 μ F) are connected at the input and output of the 7805 to ensure stability and reduce noise. The Arduino reads this reference voltage and uses it for calibration purposes, correcting any variations in ADC readings caused by supply fluctuations. The 7805 circuit is designed to provide a stable and low-noise reference, making it suitable for instrumentation applications. High-value resistors are used in voltage divider circuits to minimize current flow and reduce power loss. Capacitors are included in all sensing circuits to filter noise and improve signal stability. Proper grounding is essential to ensure accurate measurements. All sensor grounds are connected to a common ground to avoid measurement errors due to potential differences. The sensors are designed to be non-intrusive, meaning they do not affect the actual operation of the system while providing accurate measurements.



1. Power Section:-Input supply connected to buck and boost converters, Converter outputs connected to relay module, Relay selects required voltage (step-down or step-up)

4. Voltage Sensing Circuit:- Voltage divider used for Input voltage (V_{in}), Output voltage (V_{out}), Reduces high voltage to safe ADC levels and Capacitors used for filtering noise

5. Noise Sensing Circuit:- RC (resistor-capacitor) network is used, Connected to output node and detects fluctuations and ripple

6. Reference Voltage Circuit:- Uses 7805 voltage regulator, Provides stable reference voltage, Includes capacitors for stability and used for calibration and accurate measurement

7. Display & Indicators:- LCD connected using I2C (SDA, SCL), LEDs → status indication and Buzzer → fault alert

8. User Input Section:- Potentiometer → Analog input (user control), Push button → digital input (mode/function selection)

9. Grounding:- All components share common ground, Ensures proper signal reference and stable operation

Circuit Operation

1. Power Generation & Selection:- Buck/boost converters generate required voltage, Relay selects appropriate voltage based on condition

2. Load Control:- MOSFET controls load using PWM, Provides efficient power control with low loss

3. Continuous Monitoring:- Voltage dividers measure V_{in} and V_{out} , Noise circuit detects ripple Capacitors improve signal stability

4. Calibration & Accuracy:- 7805 provides stable reference voltage, Used to improve measurement accuracy

5. Control Processing:- Microcontroller processes all inputs Controls:

- Relay switching
- MOSFET operation
- Display and alerts

6. User Interaction:- Potentiometer + push button used for control, LCD displays system parameters clearly

7. Safety & Reliability:- Proper grounding maintained, Filtering reduces noise and Ensures safe and stable operation

CHAPTER 6

6.1 ADVANTAGES

1. **High Accuracy:-** Uses reference voltage and calibration, Gives correct and stable output even if input fluctuates
2. **Real-Time Monitoring:-** Displays input voltage, output voltage, and noise ,Helps in understanding system performance and power quality
3. **Smooth PWM Control:-** Uses PWM (Pulse Width Modulation) for power control, Provides soft start → protects devices from sudden surge
4. **Automatic & Manual Modes:-** Can work automatically or manually, Gives flexibility based on user requirement
5. **Simple Hardware Design:-** Uses single potentiometer + one push button, Reduces circuit complexity and cost
6. **User-Friendly Interface:-** LCD display shows system data clearly, LEDs indicate status and Buzzer gives alert during faults
7. **Safety Features:-** Alerts user during abnormal conditions, Prevents damage to connected devices
8. **Modular Design:-** Easy to upgrade or modify, Future features like IoT, data logging can be added
9. **Compact and Efficient:-** Less components → small size, Better efficiency compared to traditional stabilizers
10. **Reliable Performance:-** Works consistently under different voltage conditions, Suitable for modern applications

6.2 APPLICATIONS

1. **Domestic Power Supply:-** Provides stable voltage for home appliances, Protects devices like computers, TVs, and communication equipment
2. **Industrial Applications:-** Used in machinery and automation systems, Prevents performance issues and improves safety
3. **Sensitive Electronic Devices:-** Suitable for control systems and precision instruments, Ensures proper functioning without voltage disturbance
4. **Renewable Energy Systems:-** Used in solar power systems, Handles voltage variations due to weather changes

5. Battery-Based Systems:- Maintains stable output in battery-powered setups, Improves efficiency and battery life

6. Laboratories & Research:- Works as a power conditioning unit, Useful for experiments with monitoring and calibration

7. Educational Purpose:- Helps students learn power electronics and embedded systems, Provides practical hands-on experience

8. Automotive Applications:- Can be used in vehicles for stable power supply, Supports electronic control units and accessories

9. Portable Electronics:- Suitable for portable devices requiring regulated voltage, Ensures efficient power management

10. Versatile Usage:- Can be adapted for different applications, Suitable for both practical and research purposes

6.3 LIMITATIONS

1. Power Loss in Regulator:- Uses 7805 linear regulator, Produces heat at higher input voltage → reduces efficiency

2. Measurement Errors:- Uses voltage divider circuits, Errors due to resistor tolerance and ADC limitations

3. Limited Processing Power:- Arduino UNO has less memory and speed, Cannot support advanced algorithms or large data logging

4. Relay Switching Issues:- Mechanical relay causes delay, Wears out over time → reduces reliability

5. Limited Voltage Range:- Supports only fixed ranges (12V, 24V), Not suitable for fine voltage control

6. Basic Noise Measurement:- Uses simple sampling method, Cannot perform accurate frequency analysis

7. No Temperature Monitoring:- Cannot detect overheating of MOSFET or regulator, Risk of thermal damage

8. Limited Data Storage:- EEPROM memory is small, Can store only limited data

9. Low to Medium Power Only:- Not suitable for high-power industrial loads, Needs additional protection for heavy loads

10. User Interface Complexity:- Single potentiometer + push button used for many functions, May be confusing for new users

11. Scope for Improvement:-Can be improved with better sensors, controllers, and communication systems

6.4 FUTURE SCOPE

1. IoT Integration:- Add Wi-Fi / Bluetooth modules, Send data (voltage, noise, status) to cloud and Enable remote monitoring and control via mobile/web

2. AI-Based Control:- Use Artificial Intelligence instead of fixed settings, Analyze past data to make smart decisions and Automatically adjust voltage, PWM, and load

3. Predictive Maintenance:- Monitor system continuously, Detect abnormal patterns early and Alert user before failure occurs

4. Smart Fault Detection:- Identify faults like Overvoltage, Undervoltage, Load issues and Provide suggestions for correction

5. Cloud Data Logging:- Replace EEPROM with cloud storage, Store large data for long time and Enable data analysis and performance tracking

6. Smart Home / Industrial Integration:- Connect with automation systems, Coordinate with other devices and Optimize overall power usage

7. Mobile App & Voice Control:- Control system using smartphone, Monitor status remotely and Use voice commands for operation

8. Energy Optimization:- Use AI to reduce power loss, Adjust system for maximum efficiency

9. Intelligent Automation:- System becomes self-operating, No need for constant manual control

CHAPTER 7

7.1 FINAL CONCLUSION

The smart servo-based voltage stabilizer successfully demonstrates an advanced approach to voltage regulation. It ensures stable and reliable output voltage under varying input conditions. The integration of sensing, control, and protection enhances system performance. Real-time monitoring of voltage and noise improves power quality analysis. The use of PWM-based MOSFET control enables smooth and efficient power delivery. Soft-start functionality protects connected loads from sudden surges. The inclusion of a reference voltage improves measurement accuracy. Fault detection mechanisms ensure safety and prevent damage to devices. The system supports both automatic and manual operation modes. User interaction is simplified using minimal hardware components. The LCD display and indicators provide clear system status information. The modular design allows easy expansion and future improvements. The project combines power electronics, embedded systems, and instrumentation effectively. It overcomes many limitations of conventional stabilizers. Overall, the system is efficient, reliable, and suitable for modern applications.